Nexus - SMC Order Flow Confluence Engine

```
// © AlphaX
//@version=6
indicator("[ A L P H A X ] Nexus - SMC Order Flow Confluence Engine", shorttitle="AlphaX Nexus", overlay=true,
  max_labels_count=500, max_bars_back=500, max_boxes_count=500, max_lines_count=500)

// ══════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════
// [redacted] ALPHAX NEXUS [redacted]
// [redacted] Smart Money Concepts × Order Flow Confluence Engine [redacted]
// ══════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════


// ══════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════
// ═══════════════════════════════════════════════════════════ USER INPUTS ═════════════════════════════════════════════════════════════════════════════
// ══════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════

// ─── Structure Settings (tuned for XAUUSD / Gold) ─────────────────────────────────────────────────────────
swingLen        = input.int(7,     "Swing Detection Length",      minval=2,   maxval=20,  group="Market Structure")
showBOS         = input.bool(true, "Show Break of Structure",                             group="Market Structure")
showCHoCH       = input.bool(true, "Show Change of Character",                            group="Market Structure")
showStructLabel = input.bool(true, "Show Structure Labels",                               group="Market
```

```
Structure")

// ——— Order Block Settings (tuned for Gold volatility)
————————————————————————————————

showOB          = input.bool(true, "Show Order Blocks",                    group="Order Blocks")
obMaxBoxes      = input.int(4,     "Max Active Order Blocks",    minval=1,   maxval=15,   group="Order
Blocks")
obMitigateMode  = input.string("Wick", "Mitigation Mode", options=["Wick", "Close"],
group="Order Blocks")
obVolumeFilter  = input.bool(true, "Require Volume Confirmation",          group="Order
Blocks")
obVolMultiplier = input.float(1.4, "OB Volume Multiplier",       minval=0.5, maxval=3.0, step=0.1,
group="Order Blocks")
obMaxAge        = input.int(80,    "OB Max Age (bars)",         minval=20,  maxval=500,  group="Order
Blocks")

// ——— Fair Value Gap Settings (tuned for Gold spreads)
————————————————————————————————

showFVG         = input.bool(true, "Show Fair Value Gaps",                 group="Fair Value
Gaps")
fvgMaxBoxes     = input.int(4,     "Max Active FVGs",           minval=1,   maxval=15,   group="Fair
Value Gaps")
fvgMinSize      = input×float(15.0,"Min FVG Size (% of ATR)",   minval=0.0, maxval=100.0, step=5.0,
group="Fair Value Gaps")
fvgMaxAge       = input.int(60,    "FVG Max Age (bars)",        minval=10,  maxval=500,  group="Fair
Value Gaps")

// ——— Liquidity Sweep Settings (tuned for Gold manipulation)
————————————————————————————

showLiqSweep    = input.bool(true,  "Show Liquidity Sweeps",               group="Liquidity")
liqLookback     = input.int(25,     "Liquidity Lookback",        minval=5,   maxval=100,
group="Liquidity")
showEqHL        = input.bool(true,  "Show Equal Highs/Lows",               group="Liquidity")
eqThreshold     = input.float(0.08, "Equal Level Threshold (%)", minval=0.01, maxval=1.0, step=0.01,
group="Liquidity")

// ——— Order Flow / Volume Analysis (tuned for Gold)
————————————————————————————————————
```

```
showVolProfile  = input.bool(true,  "Show Volume Analysis",                group="Order Flow")
volSmaLen       = input.int(20,     "Volume SMA Length",       minval=5,   maxval=100,  group="Order Flow")
volSpikeMulti   = input.float(1.8,  "Volume Spike Multiplier",  minval=1.2, maxval=5.0, step=0.1, group="Order Flow")
showAbsorption  = input×bool(true,  "Show Absorption Detection",           group="Order Flow")
absorbRatio     = input.float(2.0,  "Absorption Wick/Body Ratio", minval=1.5, maxval=5.0, step=0.1, group="Order Flow")
showDelta       = input.bool(true,  "Show Delta Divergence",               group="Order Flow")
deltaLen        = input.int(4,      "Delta Smooth Length",      minval=2,   maxval=20,   group="Order Flow")

// ——— Confluence & Signals (tuned for Gold precision) ———————————————————
minConfluence   = input.int(3,      "Min Confluence Score",     minval=1,   maxval=7,    group="Confluence Engine")
showSignals     = input×bool(true,  "Show Entry Signals",                  group="Confluence Engine")
signalCooldown  = input.int(12,     "Signal Cooldown (bars)",   minval=3,   maxval=50,   group="Confluence Engine")

// ——— EMA Settings (tuned for Gold trends) ——————————————————————————————
showEmaFast     = input.bool(true,  "Show Fast EMA",                       group="EMA Settings")
showEmaMedium   = input.bool(true,  "Show Medium EMA",                     group="EMA Settings")
showEmaSlow     = input.bool(true,  "Show Slow EMA",                       group="EMA Settings")
emaFastLength   = input.int(21,     "Fast EMA Length",         minval=1,   maxval=500,  group="EMA Settings")
emaMediumLength = input.int(50,     "Medium EMA Length",       minval=1,   maxval=500,  group="EMA Settings")
emaSlowLength   = input.int(200,    "Slow EMA Length",         minval=1,   maxval=500,  group="EMA Settings")

// ——— Trend Cloud (tuned for Gold) ——————————————————————————————————————
showCloud       = input×bool(true,  "Show Trend Cloud",                    group="Trend Cloud")
```

```
cloudFastLen    = input.int(9,      "Cloud Fast Length",         minval=3,   maxval=50,   group="Trend
Cloud")
cloudSlowLen    = input.int(26,     "Cloud Slow Length",         minval=10,  maxval=100,  group="Trend
Cloud")

//
════════════════════════════════════════════════════════════════════════════════
══════════════════════════════════════════════════
// ════════════════════════════════════════════ THEME COLORS
══════════════════════════════════════════════════════════════
//
════════════════════════════════════════════════════════════════════════════════
══════════════════════════════════════════════════

color BULL_PRIMARY    = input.color(#c8e624, "Bull Primary Color",    group="Theme")
color BULL_BRIGHT     = input.color(#d4f03a, "Bull Bright Color",     group="Theme")
color BULL_DIM        = input.color(#9ab81c, "Bull Dim Color",        group="Theme")
color BEAR_PRIMARY    = input.color(#ff1744, "Bear Primary Color",    group="Theme")
color BEAR_BRIGHT     = input.color(#ff5252, "Bear Bright Color",     group="Theme")
color BEAR_DIM        = input.color(#d50032, "Bear Dim Color",        group="Theme")
color NEUTRAL_COLOR   = input×color(#555555, "Neutral / Inactive",    group="Theme")
color OB_BULL_COLOR   = input×color(#c8e624, "Bullish OB Color",      group="Theme")
color OB_BEAR_COLOR   = input×color(#ff1744, "Bearish OB Color",      group="Theme")
color FVG_BULL_COLOR  = input×color(#c8e624, "Bullish FVG Color",     group="Theme")
color FVG_BEAR_COLOR  = input×color(#ff1744, "Bearish FVG Color",     group="Theme")
color LIQ_COLOR       = input×color(#888888, "Liquidity Line Color", group="Theme")
color EMA_FAST_CLR    = input.color(#c8e624, "Fast EMA Color",        group="Theme")
color EMA_MED_CLR     = input.color(#888888, "Medium EMA Color",      group="Theme")
color EMA_SLOW_CLR    = input.color(#555555, "Slow EMA Color",        group="Theme")


//
════════════════════════════════════════════════════════════════════════════════
══════════════════════════════════════════════════
// ════════════════════════════════════════════ CORE CALCULATIONS
══════════════════════════════════════════════════════
//
════════════════════════════════════════════════════════════════════════════════
```

```
═══════════════════════════════════════════════════

// ─── ATR for dynamic sizing
──────────────────────────────────────────────────────────────────────────────────────────────────
──────────
atrVal = ta×atr(14)

// ─── EMAs
──────────────────────────────────────────────────────────────────────────────────────────────────
──────────────────────────────────────────────
emaFast   = ta.ema(close, emaFastLength)
emaMedium = ta.ema(close, emaMediumLength)
emaSlow   = ta.ema(close, emaSlowLength)

emaTrendBull = emaFast > emaMedium and emaMedium > emaSlow
emaTrendBear = emaFast < emaMedium and emaMedium < emaSlow
priceAboveEma = close > emaSlow
priceBelowEma = close < emaSlow

plot(showEmaFast   ? emaFast   : na, "Fast EMA",   color=EMA_FAST_CLR, linewidth=1,
style=plot.style_cross)
plot(showEmaMedium ? emaMedium : na, "Medium EMA",  color=EMA_MED_CLR,  linewidth=1)
plot(showEmaSlow   ? emaSlow   : na, "Slow EMA",    color=EMA_SLOW_CLR, linewidth=2)

// ─── Trend Cloud (Kijun/Tenkan style for structure context)
──────────────────────────────────────────
cloudFast = (ta×highest(high, cloudFastLen) + ta.lowest(low, cloudFastLen)) / 2
cloudSlow = (ta.highest(high, cloudSlowLen) + ta.lowest(low, cloudSlowLen)) / 2

cloudBull = cloudFast > cloudSlow
cloudBear = cloudFast < cloudSlow

pCloudFast = plot(showCloud ? cloudFast : na, "Cloud Fast", color=color×new(cloudBull ?
BULL_PRIMARY : BEAR_PRIMARY, 60), linewidth=1)
pCloudSlow = plot(showCloud ? cloudSlow : na, "Cloud Slow", color=color×new(cloudBull ?
BULL_DIM : BEAR_DIM, 60), linewidth=1)
fill(pCloudFast, pCloudSlow,
  color=showCloud ? color.new(cloudBull ? BULL_PRIMARY : BEAR_PRIMARY, 88) : na,
```

```
  title="Trend Cloud Fill")

// ════════════════════════════════════════════════════════════════════════════════════════════════════════════
// ═══════════════════════════════════════════════ VOLUME / ORDER FLOW ENGINE ═══════════════════════════════════════════
// ════════════════════════════════════════════════════════════════════════════════════════════════════════════

volSma       = ta×sma(volume, volSmaLen)
volSpike     = volume > volSma * volSpikeMulti
volAboveAvg  = volume > volSma * 1.0
relVol       = volSma > 0 ? volume / volSma : 1.0

// ── Estimated Delta (Buy vs Sell Volume Estimation) ──────────────────────────────────────────────────────
// Without tick data, we estimate using candle position method
bodySize     = math.abs(close - open)
candleRange  = high - low
upperWick    = high - math.max(close, open)
lowerWick    = math.min(close, open) - low

// Buy volume estimation: proportion of range that price moved up
buyVolEst    = candleRange > 0 ? volume * ((close - low) / candleRange) : volume * 0.5
sellVolEst   = volume - buyVolEst
rawDelta     = buyVolEst - sellVolEst
smoothDelta  = ta.ema(rawDelta, deltaLen)

// Delta divergence: price makes new high but delta is declining (distribution)
// or price makes new low but delta is rising (accumulation)
deltaRising  = smoothDelta > smoothDelta[1] and smoothDelta > smoothDelta[2]
deltaFalling = smoothDelta < smoothDelta[1] and smoothDelta < smoothDelta[2]

priceNewHigh = high == ta.highest(high, 10)
priceNewLow  = low  == ta.lowest(low, 10)
```

```
deltaBearDiv = priceNewHigh and deltaFalling and showDelta  // Distribution
deltaBullDiv = priceNewLow  and deltaRising  and showDelta  // Accumulation

// ─── Absorption Detection ───────────────────────────────────────────────────────────────────
// High volume + large wick + small body = institutional absorption
bullAbsorption = showAbsorption and
  lowerWick > bodySize * absorbRatio and
  volAboveAvg and
  close > open and
  lowerWick > upperWick * 1.5

bearAbsorption = showAbsorption and
  upperWick > bodySize * absorbRatio and
  volAboveAvg and
  close < open and
  upperWick > lowerWick * 1.5

// ─── Volume Climax (Exhaustion) ───────────────────────────────────────────────────────────
volClimax    = volume > volSma * (volSpikeMulti * 1.5)
bullExhaust  = volClimax and close < open and upperWick > bodySize  // Selling climax at top
bearExhaust  = volClimax and close > open and lowerWick > bodySize  // Buying climax at bottom

// ═══════════════════════════════════════════════════════════════════════════════════════════
// ═══════════════════════════════ SWING DETECTION ═══════════════════════════════════════
// ═══════════════════════════════════════════════════════════════════════════════════════════

swingHigh = ta.pivothigh(high, swingLen, swingLen)
swingLow  = ta.pivotlow(low,   swingLen, swingLen)
```

```
var float lastSwingHigh     = na
var float lastSwingLow      = na
var int   lastSwingHighBar  = na
var int   lastSwingLowBar   = na
var float prevSwingHigh     = na
var float prevSwingLow      = na
var int   prevSwingHighBar  = na
var int   prevSwingLowBar   = na

if not na(swingHigh)
    prevSwingHigh    := lastSwingHigh
    prevSwingHighBar := lastSwingHighBar
    lastSwingHigh    := swingHigh
    lastSwingHighBar := bar_index - swingLen

if not na(swingLow)
    prevSwingLow    := lastSwingLow
    prevSwingLowBar := lastSwingLowBar
    lastSwingLow    := swingLow
    lastSwingLowBar := bar_index - swingLen

//
═══════════════════════════════════════════════════════════════════════════════════════
═══════════════════════════════════════
// ═══════════════════════════════════ MARKET STRUCTURE (BOS / CHoCH)
═══════════════════════════
//
═══════════════════════════════════════════════════════════════════════════════════════
═══════════════════════════════════════

var int structureTrend = 0  // 1 = bullish, -1 = bearish, 0 = undefined

// Break of Structure: continuation break
// CHoCH: first break against the trend (character change)

bool bosUp   = false
bool bosDown = false
bool chochUp   = false
```

```
bool chochDown = false

// Bullish BOS: price breaks above the last swing high in a bullish trend
if not na(lastSwingHigh) and close > lastSwingHigh and close[1] <= lastSwingHigh
    if structureTrend >= 0
        bosUp := true
        structureTrend := 1
    else
        chochUp := true
        structureTrend := 1

// Bearish BOS: price breaks below the last swing low in a bearish trend
if not na(lastSwingLow) and close < lastSwingLow and close[1] >= lastSwingLow
    if structureTrend <= 0
        bosDown := true
        structureTrend := -1
    else
        chochDown := true
        structureTrend := -1

// ─── Structure Labels
────────────────────────────────────────────────────────────────────────
────────────────────

if showBOS and bosUp and showStructLabel
    label.new(bar_index, low,
      text      = "BOS ↑",
      yloc      = yloc.belowbar,
      color     = color×new(BULL_PRIMARY, 30),
      textcolor = #1a1a1a,
      size      = size.tiny,
      style     = label.style_label_up)

if showBOS and bosDown and showStructLabel
    label.new(bar_index, high,
      text      = "BOS ↓",
      yloc      = yloc.abovebar,
      color     = color×new(BEAR_PRIMARY, 30),
      textcolor = #ffffff,
```

```
      size      = size.tiny,
      style     = label.style_label_down)

if showCHoCH and chochUp and showStructLabel
    label.new(bar_index, low,
      text      = "CHoCH ↑",
      yloc      = yloc.belowbar,
      color     = color×new(BULL_BRIGHT, 10),
      textcolor = #1a1a1a,
      size      = size.small,
      style     = label.style_label_up)

if showCHoCH and chochDown and showStructLabel
    label.new(bar_index, high,
      text      = "CHoCH ↓",
      yloc      = yloc.abovebar,
      color     = color×new(BEAR_BRIGHT, 10),
      textcolor = #ffffff,
      size      = size.small,
      style     = label.style_label_down)

//
═══════════════════════════════════════════════════════════════════════════════
═══════════════════════════════════════════
// ═══════════════════════════════════════ ORDER BLOCKS
═══════════════════════════════════════════════════════════
//
═══════════════════════════════════════════════════════════════════════════════
═══════════════════════════════════════════

// Order Block: the last opposing candle before an impulsive move that breaks structure
// Bullish OB: last bearish candle before a bullish BOS
// Bearish OB: last bullish candle before a bearish BOS

type OrderBlock
    box     bx
    float   top
    float   bottom
```

```
    int     startBar
    bool    isBull
    bool    active

var array<OrderBlock> obArray = array.new<OrderBlock>()

// Find the last bearish candle before bullish break
findBullOB() =>
    float obTop    = na
    float obBottom = na
    int   obBar    = na
    bool  volOK    = true
    for i = 1 to 15
        if close[i] < open[i]  // bearish candle
            obTop    := high[i]
            obBottom := low[i]
            obBar    := bar_index - i
            volOK    := not obVolumeFilter or volume[i] > volSma[i] * obVolMultiplier
            break
    [obTop, obBottom, obBar, volOK]

// Find the last bullish candle before bearish break
findBearOB() =>
    float obTop    = na
    float obBottom = na
    int   obBar    = na
    bool  volOK    = true
    for i = 1 to 15
        if close[i] > open[i]  // bullish candle
            obTop    := high[i]
            obBottom := low[i]
            obBar    := bar_index - i
            volOK    := not obVolumeFilter or volume[i] > volSma[i] * obVolMultiplier
            break
    [obTop, obBottom, obBar, volOK]

// Create Order Blocks on structure breaks
if showOB and (bosUp or chochUp)
```

```
    [obT, obB, obBr, volCheck] = findBullOB()
    if not na(obT) and volCheck
        // Limit active OBs
        if array.size(obArray) >= obMaxBoxes
            oldest = array.shift(obArray)
            box.delete(oldest.bx)
        newOB = OrderBlock.new(
          bx       = box.new(obBr, obT, bar_index + 5, obB,
                     border_color = color×new(OB_BULL_COLOR, 40),
                     bgcolor      = color.new(OB_BULL_COLOR, 88),
                     border_width = 1,
                     border_style = line.style_solid,
                     extend       = extend×right),
          top      = obT,
          bottom   = obB,
          startBar = obBr,
          isBull   = true,
          active   = true)
        array.push(obArray, newOB)

if showOB and (bosDown or chochDown)
    [obT, obB, obBr, volCheck] = findBearOB()
    if not na(obT) and volCheck
        if array.size(obArray) >= obMaxBoxes
            oldest = array.shift(obArray)
            box.delete(oldest.bx)
        newOB = OrderBlock.new(
          bx       = box.new(obBr, obT, bar_index + 5, obB,
                     border_color = color×new(OB_BEAR_COLOR, 40),
                     bgcolor      = color.new(OB_BEAR_COLOR, 88),
                     border_width = 1,
                     border_style = line.style_solid,
                     extend       = extend×right),
          top      = obT,
          bottom   = obB,
          startBar = obBr,
          isBull   = false,
          active   = true)
```

```
        array.push(obArray, newOB)

// ——— Order Block Mitigation & Age Management
————————————————————————————————————————————————————

var bool priceInBullOB = false
var bool priceInBearOB = false
priceInBullOB := false
priceInBearOB := false

if array.size(obArray) > 0
    for i = array.size(obArray) - 1 to 0
        ob = array.get(obArray, i)
        if ob.active
            // Check age
            if bar_index - ob.startBar > obMaxAge
                ob.active := false
                box.delete(ob.bx)
                array.remove(obArray, i)
                continue

            // Check mitigation
            mitigated = false
            if ob.isBull
                if obMitigateMode == "Close"
                    mitigated := close < ob.bottom
                else
                    mitigated := low < ob.bottom
                // Check if price is currently IN the bullish OB (potential long entry zone)
                if low <= ob.top and high >= ob.bottom and not mitigated
                    priceInBullOB := true
            else
                if obMitigateMode == "Close"
                    mitigated := close > ob.top
                else
                    mitigated := high > ob.top
                // Check if price is currently IN the bearish OB (potential short entry zone)
                if high >= ob.bottom and low <= ob.top and not mitigated
                    priceInBearOB := true
```

```
        if mitigated
            ob.active := false
            box.delete(ob.bx)
            array.remove(obArray, i)

//
═══════════════════════════════════════════════════════════════════════════════════════════════
// ═══════════════════════════════ FAIR VALUE GAPS
═══════════════════════════════════════════════════════
//
═══════════════════════════════════════════════════════════════════════════════════════════════

type FairValueGap
    box     bx
    float   top
    float   bottom
    int     startBar
    bool    isBull
    bool    active

var array<FairValueGap> fvgArray = array.new<FairValueGap>()

// Bullish FVG: gap between candle[2] high and candle[0] low (current candle)
// The middle candle is the impulse
fvgMinATR     = atrVal × fvgMinSize / 100.0
bullFvgTop    = low
bullFvgBottom = high[2]
bullFvgExists = bullFvgBottom < bullFvgTop and
  close[1] > open[1] and
  (bullFvgTop - bullFvgBottom) > fvgMinATR

bearFvgTop    = low[2]
bearFvgBottom = high
bearFvgExists = bearFvgTop > bearFvgBottom and
  close[1] < open[1] and
```

```
  (bearFvgTop - bearFvgBottom) > fvgMinATR

if showFVG and bullFvgExists
    if array.size(fvgArray) >= fvgMaxBoxes
        oldest = array.shift(fvgArray)
        box.delete(oldest.bx)
    newFVG = FairValueGap.new(
      bx       = box.new(bar_index - 1, bullFvgTop, bar_index + 3, bullFvgBottom,
                 border_color = color×new(FVG_BULL_COLOR, 50),
                 bgcolor      = color.new(FVG_BULL_COLOR, 92),
                 border_width = 1,
                 border_style = line.style_dotted,
                 extend       = extend×right),
      top      = bullFvgTop,
      bottom   = bullFvgBottom,
      startBar = bar_index - 1,
      isBull   = true,
      active   = true)
    array.push(fvgArray, newFVG)

if showFVG and bearFvgExists
    if array.size(fvgArray) >= fvgMaxBoxes
        oldest = array.shift(fvgArray)
        box.delete(oldest.bx)
    newFVG = FairValueGap.new(
      bx       = box.new(bar_index - 1, bearFvgTop, bar_index + 3, bearFvgBottom,
                 border_color = color×new(FVG_BEAR_COLOR, 50),
                 bgcolor      = color.new(FVG_BEAR_COLOR, 92),
                 border_width = 1,
                 border_style = line.style_dotted,
                 extend       = extend.right),
      top      = bearFvgTop,
      bottom   = bearFvgBottom,
      startBar = bar_index - 1,
      isBull   = false,
      active   = true)
    array.push(fvgArray, newFVG)
```

```
// ─── FVG Fill Detection & Age Management
──────────────────────────────────────────────────────────────────

var bool priceInBullFVG = false
var bool priceInBearFVG = false
priceInBullFVG := false
priceInBearFVG := false

if array.size(fvgArray) > 0
    for i = array.size(fvgArray) - 1 to 0
        fvg = array.get(fvgArray, i)
        if fvg.active
            // Age check
            if bar_index - fvg.startBar > fvgMaxAge
                fvg.active := false
                box.delete(fvg.bx)
                array.remove(fvgArray, i)
                continue

            // Fill check
            filled = false
            if fvg.isBull
                filled := low <= fvg.bottom
                if low <= fvg.top and high >= fvg.bottom and not filled
                    priceInBullFVG := true
            else
                filled := high >= fvg.top
                if high >= fvg.bottom and low <= fvg.top and not filled
                    priceInBearFVG := true

            if filled
                fvg.active := false
                box.delete(fvg.bx)
                array.remove(fvgArray, i)

//
═══════════════════════════════════════════════════════════════════════════
══════════════════════════════════════════════
// ══════════════════════════════════════════ LIQUIDITY SWEEP DETECTION
```

```
═══════════════════════════════════════
//
═══════════════════════════════════════════════════════════════════════════════
═══════════════════════════════════════

// Detect when price sweeps a recent swing high/low then reverses
recentHigh = ta.highest(high, liqLookback)
recentLow  = ta.lowest(low,   liqLookback)

// Bullish liquidity sweep: price dips below recent lows then closes back above
bullLiqSweep = low < recentLow[1] and close > recentLow[1] and close > open and showLiqSweep
// Bearish liquidity sweep: price spikes above recent highs then closes back below
bearLiqSweep = high > recentHigh[1] and close < recentHigh[1] and close < open and
showLiqSweep

// ─── Equal Highs / Equal Lows (Liquidity Pools)
──────────────────────────────────────────────────────────

var float eqHighLevel = na
var float eqLowLevel  = na
var line  eqHighLine  = na
var line  eqLowLine   = na

if showEqHL and not na(lastSwingHigh) and not na(prevSwingHigh)
    pctDiff = math.abs(lastSwingHigh - prevSwingHigh) / prevSwingHigh * 100
    if pctDiff < eqThreshold and lastSwingHighBar != prevSwingHighBar
        eqHighLevel := (lastSwingHigh + prevSwingHigh) / 2
        if not na(eqHighLine)
            line.delete(eqHighLine)
        eqHighLine := line.new(
          x1    = math.max(0, prevSwingHighBar),
          y1    = eqHighLevel,
          x2    = bar_index + 10,
          y2    = eqHighLevel,
          color = color×new(LIQ_COLOR, 30),
          style = line.style_dashed,
          width = 1)

if showEqHL and not na(lastSwingLow) and not na(prevSwingLow)
```

```
    pctDiff = math.abs(lastSwingLow - prevSwingLow) / prevSwingLow * 100
    if pctDiff < eqThreshold and lastSwingLowBar != prevSwingLowBar
        eqLowLevel := (lastSwingLow + prevSwingLow) / 2
        if not na(eqLowLine)
            line.delete(eqLowLine)
        eqLowLine := line.new(
          x1    = math.max(0, prevSwingLowBar),
          y1    = eqLowLevel,
          x2    = bar_index + 10,
          y2    = eqLowLevel,
          color = color×new(LIQ_COLOR, 30),
          style = line.style_dashed,
          width = 1)

// ════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════
// ══════════════════════════════════════════════════════ CONFLUENCE ENGINE ══════════════════════════════════════════════════════════
// ════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════

// Each confluence factor adds to the score
// Maximum possible score: 8 per direction

// ── Bullish Confluence ─────────────────────────────────────────────────────────────────────────────────────────────
int bullScore = 0

// 1. Market Structure: bullish (BOS up or CHoCH up recently, or structureTrend == 1)
bullScore += structureTrend == 1 ? 1 : 0

// 2. Price in Bullish Order Block
bullScore += priceInBullOB ? 1 : 0

// 3. Price in Bullish FVG
```

```
bullScore += priceInBullFVG ? 1 : 0

// 4. Liquidity Sweep below (bullish sweep)
bullScore += bullLiqSweep ? 1 : 0

// 5. Volume Confirmation (above average on bullish candle)
bullScore += (volAboveAvg and close > open) ? 1 : 0

// 6. Bullish Absorption (institutional buying detected)
bullScore += bullAbsorption ? 1 : 0

// 7. Delta Accumulation Divergence
bullScore += deltaBullDiv ? 1 : 0

// 8. EMA Trend Alignment
bullScore += emaTrendBull or (priceAboveEma and emaFast > emaMedium) ? 1 : 0

// ─── Bearish Confluence ────────────────────────────────────────────────────────────────────────
int bearScore = 0

// 1. Market Structure: bearish
bearScore += structureTrend == -1 ? 1 : 0

// 2. Price in Bearish Order Block
bearScore += priceInBearOB ? 1 : 0

// 3. Price in Bearish FVG
bearScore += priceInBearFVG ? 1 : 0

// 4. Liquidity Sweep above (bearish sweep)
bearScore += bearLiqSweep ? 1 : 0

// 5. Volume Confirmation
bearScore += (volAboveAvg and close < open) ? 1 : 0

// 6. Bearish Absorption
```

```
bearScore += bearAbsorption ? 1 : 0

// 7. Delta Distribution Divergence
bearScore += deltaBearDiv ? 1 : 0

// 8. EMA Trend Alignment
bearScore += emaTrendBear or (priceBelowEma and emaFast < emaMedium) ? 1 : 0

//
═══════════════════════════════════════════════════════════════════════════════
// ═══════════════════════════ CONFIDENCE CALCULATION ═══════════════════════════
//
═══════════════════════════════════════════════════════════════════════════════

calcBullConfidence() =>
    float c = 0.0
    // Structure weight (25%)
    c += structureTrend == 1 ? 25.0 : structureTrend == 0 ? 10.0 : 0.0
    // POI (Order Block + FVG) weight (25%)
    c += priceInBullOB ? 15.0 : 0.0
    c += priceInBullFVG ? 10.0 : 0.0
    // Liquidity weight (15%)
    c += bullLiqSweep ? 15.0 : 0.0
    // Order Flow weight (25%)
    c += bullAbsorption ? 10.0 : 0.0
    c += deltaBullDiv ? 8.0 : 0.0
    c += (volAboveAvg and close > open) ? 7.0 : 0.0
    // Trend weight (10%)
    c += emaTrendBull ? 10.0 : (priceAboveEma ? 5.0 : 0.0)
    math.min(100.0, c)

calcBearConfidence() =>
    float c = 0.0
    c += structureTrend == -1 ? 25.0 : structureTrend == 0 ? 10.0 : 0.0
    c += priceInBearOB ? 15.0 : 0.0
```

```
    c += priceInBearFVG ? 10.0 : 0.0
    c += bearLiqSweep ? 15.0 : 0.0
    c += bearAbsorption ? 10.0 : 0.0
    c += deltaBearDiv ? 8.0 : 0.0
    c += (volAboveAvg and close < open) ? 7.0 : 0.0
    c += emaTrendBear ? 10.0 : (priceBelowEma ? 5.0 : 0.0)
    math.min(100.0, c)

bullConfidence = calcBullConfidence()
bearConfidence = calcBearConfidence()

//
══════════════════════════════════════════════════════════════════════════════════
══════════════════════════════════════════════════
// ═════════════════════════════════════════════ SIGNAL GENERATION
═══════════════════════════════════════════════════════
//
══════════════════════════════════════════════════════════════════════════════════
══════════════════════════════════════════════════

var int lastSignalBar = 0

bool cooldownOK = (bar_index - lastSignalBar) >= signalCooldown

// Entry conditions: minimum confluence + cooldown
bool bullSignal = showSignals and bullScore >= minConfluence and cooldownOK and close > open
bool bearSignal = showSignals and bearScore >= minConfluence and cooldownOK and close < open

// Prevent simultaneous signals
if bullSignal and bearSignal
    if bullScore > bearScore
        bearSignal := false
    else if bearScore > bullScore
        bullSignal := false
    else
        bullSignal := false
        bearSignal := false
```

```
if bullSignal or bearSignal
    lastSignalBar := bar_index

// ─── Confidence Grade
──────────────────────────────────────────────────────────────────────────────────────────────

getGrade(float conf) =>
    conf >= 75 ? "A+" :
      conf >= 60 ? "A" :
      conf >= 45 ? "B" :
      conf >= 30 ? "C" : "D"

getGradeColor(float conf, bool isBull) =>
    if conf >= 75
        isBull ? BULL_BRIGHT : BEAR_BRIGHT
    else if conf >= 60
        isBull ? BULL_PRIMARY : BEAR_PRIMARY
    else if conf >= 45
        isBull ? BULL_DIM : BEAR_DIM
    else
        NEUTRAL_COLOR

//
══════════════════════════════════════════════════════════════════════════════════════════════
// ══════════════════════════════════════════════ VISUAL OUTPUT
══════════════════════════════════════════════════════════════
//
══════════════════════════════════════════════════════════════════════════════════════════════

// ─── Entry Signals (offset labels with dotted connector lines)
────────────────────────────────

signalOffset = input×float(3.4, "Signal Label Offset (x ATR)", minval=1.0, maxval=8.0, step=0.5,
group="Display")

if bullSignal
    grade    = getGrade(bullConfidence)
```

```
    gradeClr = getGradeColor(bullConfidence, true)

    // Build confluence detail string
    string details = ""
    details += structureTrend == 1       ? "✓STR " : ""
    details += priceInBullOB             ? "✓OB "  : ""
    details += priceInBullFVG            ? "✓FVG " : ""
    details += bullLiqSweep              ? "✓LIQ " : ""
    details += (volAboveAvg and close > open) ? "✓VOL " : ""
    details += bullAbsorption            ? "✓ABS " : ""
    details += deltaBullDiv              ? "✓DLT " : ""
    details += (emaTrendBull or (priceAboveEma and emaFast > emaMedium)) ? "✓EMA " : ""

    labelText = "▲ L O N G" +
      "\n" + grade + " | " + str.tostring(math.round(bullConfidence)) + "%" +
      " | " + str.tostring(bullScore) + "/8" +
      "\n" + details

    // Calculate offset position
    float labelY = low - (atrVal × signalOffset)

    // Draw dotted connector line from candle to label
    line.new(bar_index, low, bar_index, labelY,
      color = color×new(gradeClr, 40),
      style = line.style_dotted,
      width = 1)

    // Draw the label at offset position
    label.new(bar_index, labelY,
      text      = labelText,
      color     = color×new(gradeClr, 10),
      textcolor = #1a1a1a,
      size      = size.small,
      style     = label.style_label_up)

if bearSignal
    grade    = getGrade(bearConfidence)
    gradeClr = getGradeColor(bearConfidence, false)
```

```
    string details = ""
    details += structureTrend == -1      ? "✓STR " : ""
    details += priceInBearOB             ? "✓OB "  : ""
    details += priceInBearFVG            ? "✓FVG " : ""
    details += bearLiqSweep              ? "✓LIQ " : ""
    details += (volAboveAvg and close < open) ? "✓VOL " : ""
    details += bearAbsorption            ? "✓ABS " : ""
    details += deltaBearDiv              ? "✓DLT " : ""
    details += (emaTrendBear or (priceBelowEma and emaFast < emaMedium)) ? "✓EMA " : ""

    labelText = "▼  S H O R T" +
      "\n" + grade + " | " + str.tostring(math.round(bearConfidence)) + "%" +
      " | " + str.tostring(bearScore) + "/8" +
      "\n" + details

    // Calculate offset position
    float labelY = high + (atrVal × signalOffset)

    // Draw dotted connector line from candle to label
    line.new(bar_index, high, bar_index, labelY,
      color = color×new(gradeClr, 40),
      style = line.style_dotted,
      width = 1)

    // Draw the label at offset position
    label.new(bar_index, labelY,
      text      = labelText,
      color     = color×new(gradeClr, 10),
      textcolor = #ffffff,
      size      = size.small,
      style     = label.style_label_down)

// ─── Liquidity Sweep Markers
────────────────────────────────────────────────────────────────
──────────

if bullLiqSweep
    label.new(bar_index, low,
```

```
      text      = "$ SWEEP",
      yloc      = yloc.belowbar,
      color     = color×new(BULL_DIM, 30),
      textcolor = BULL_PRIMARY,
      size      = size.tiny,
      style     = label.style_label_up)

if bearLiqSweep
    label.new(bar_index, high,
      text      = "$ SWEEP",
      yloc      = yloc.abovebar,
      color     = color.new(BEAR_DIM, 30),
      textcolor = BEAR_PRIMARY,
      size      = size.tiny,
      style     = label.style_label_down)

// ─── Absorption Markers
──────────────────────────────────────────────────────────────────────
──────────────────────

plotshape(bullAbsorption and showAbsorption,
  title    = "Bull Absorption",
  style    = shape.diamond,
  location = location.belowbar,
  color    = BULL_DIM,
  size     = size×tiny)

plotshape(bearAbsorption and showAbsorption,
  title    = "Bear Absorption",
  style    = shape.diamond,
  location = location.abovebar,
  color    = BEAR_DIM,
  size     = size×tiny)

// ─── Volume Spike Dots
──────────────────────────────────────────────────────────────────────
────────────────────────

plotshape(volSpike and showVolProfile,
  title    = "Volume Spike",
```

```
  style    = shape.circle,
  location = location.bottom,
  color    = close > open ? color.new(BULL_PRIMARY, 40) : color.new(BEAR_PRIMARY, 40),
  size     = size×tiny)

// ——— Delta Divergence Markers
————————————————————————————————————————————————————————————————————————————————
————————

plotshape(deltaBullDiv,
  title    = "Delta Accumulation",
  style    = shape.triangleup,
  location = location.belowbar,
  color    = color×new(BULL_BRIGHT, 20),
  size     = size×tiny)

plotshape(deltaBearDiv,
  title    = "Delta Distribution",
  style    = shape.triangledown,
  location = location.abovebar,
  color    = color×new(BEAR_BRIGHT, 20),
  size     = size.tiny)

//
═══════════════════════════════════════════════════════════════════════════════════
═══════════════════════════════════════════
// ═══════════════════════════════════════ DASHBOARD
═══════════════════════════════════════════════════════
//
═══════════════════════════════════════════════════════════════════════════════════
═══════════════════════════════════════════

showDash    = input.bool(true, "Show Dashboard",                                group="Display")
dashPos     = input.string("Top Right", "Dashboard Position",
  options=["Top Right", "Top Left", "Bottom Right", "Bottom Left"],             group="Display")
dashSize    = input×string("Tiny", "Dashboard Text Size",
  options=["Tiny", "Small", "Normal"],                                          group="Display")
color colDashBg      = input×color(#0a0a0a, "Dashboard Background",
group="Display")
```

```
color colTextLight   = #cccccc
color colNeutralLt   = #888888
color colDivider     = #555555

f_dashSize(sz) =>
    switch sz
        "Tiny"   => size.tiny
        "Small"  => size.small
        "Normal" => size.normal

dashTextSize = f_dashSize(dashSize)

f_dashPos(pos) =>
    switch pos
        "Top Right"    => position.top_right
        "Top Left"     => position.top_left
        "Bottom Right" => position.bottom_right
        "Bottom Left"  => position.bottom_left

// ─── Helper Functions ─────────────────────────────────────────────────────────────────────────────────────────────────

f_structIcon(trend) =>
    trend == 1 ? "▲ BULLISH" : trend == -1 ? "▼ BEARISH" : "— RANGING"

f_structClr(trend, bul, ber, neu) =>
    trend == 1 ? bul : trend == -1 ? ber : neu

f_emaTxt(bull, bear) =>
    bull ? "▲ BULL ALIGNED" : bear ? "▼ BEAR ALIGNED" : "— MIXED"

f_emaClr(bull, bear, bul, ber, neu) =>
    bull ? bul : bear ? ber : neu

f_volState(rv, spikeMulti) =>
    rv > spikeMulti * 1.5 ? "SPIKE" :
      rv > spikeMulti ? "HIGH" :
      rv > 1.0 ? "NORMAL" : "DRY"
```

```
f_volClr(rv, spikeMulti, bul, dim, neu) =>
    rv > spikeMulti * 1.5 ? bul :
      rv > spikeMulti ? dim :
      rv > 1.0 ? neu : #555555

f_deltaTxt(d) =>
    d > 0 ? "▲ BUYING" : d < 0 ? "▼ SELLING" : "— FLAT"

f_deltaClr(d, bul, ber, neu) =>
    d > 0 ? bul : d < 0 ? ber : neu

f_scoreTxt(score, conf) =>
    str.tostring(score) + " / 8  •  " + str.tostring(math.round(conf)) + "%"

f_scoreClr(score, minC, activeClr, neu) =>
    score >= minC ? activeClr : neu

f_gradeFromConf(conf) =>
    conf >= 75 ? "A+" : conf >= 60 ? "A" : conf >= 45 ? "B" : conf >= 30 ? "C" : "D"

f_gradeClr(conf, bul, dim, neu) =>
    conf >= 75 ? bul : conf >= 60 ? bul : conf >= 45 ? dim : neu

// ── Active Zone Counts ─────────────────────────────────────────────────────────────────

int dashOBCount  = 0
int dashFVGCount = 0
if array.size(obArray) > 0
    for i = 0 to array.size(obArray) - 1
        if array.get(obArray, i).active
            dashOBCount += 1
if array.size(fvgArray) > 0
    for i = 0 to array.size(fvgArray) - 1
        if array.get(fvgArray, i).active
            dashFVGCount += 1
```

```
// ── Verdict Logic ────────────────────────────────────────────────────────────────────────
string verdictTxt = "✕  NO CLEAR EDGE"
color  verdictClr = colNeutralLt

if bullScore >= 5 and structureTrend == 1 and emaTrendBull
    verdictTxt := "✓  HIGH CONFIDENCE LONG"
    verdictClr := BULL_BRIGHT
else if bearScore >= 5 and structureTrend == -1 and emaTrendBear
    verdictTxt := "✓  HIGH CONFIDENCE SHORT"
    verdictClr := BEAR_BRIGHT
else if bullScore >= minConfluence and structureTrend == 1
    verdictTxt := "△  LEAN LONG — CAUTION"
    verdictClr := BULL_DIM
else if bearScore >= minConfluence and structureTrend == -1
    verdictTxt := "▽  LEAN SHORT — CAUTION"
    verdictClr := BEAR_DIM
else if bullScore >= minConfluence and bearScore >= minConfluence
    verdictTxt := "✕  MIXED — STAND ASIDE"
    verdictClr := colNeutralLt

// ── Build Dashboard ──────────────────────────────────────────────────────────────────────
var table dash = na
if showDash
    dash := table.new(f_dashPos(dashPos), 3, 14,
      bgcolor      = color.new(colDashBg, 0),
      border_width = 1,
      border_color = color×new(colDivider, 70),
      frame_width  = 2,
      frame_color  = color.new(colDivider, 35))

if showDash and barstate.islast

    // Title uses one step larger than body text
    string titleSize = dashSize == "Tiny" ? size.small : dashSize == "Small" ? size.normal : size.large
```

```
// ─── Row 0: Title ──────────────────────────────────────────
table.cell(dash, 0, 0, "A L P H A X   N E X U S",
  text_color = BULL_PRIMARY, text_size = titleSize,
  bgcolor = color.new(colDashBg, 0))
table.cell(dash, 1, 0, "", bgcolor = color.new(colDashBg, 0))
table.cell(dash, 2, 0, "", bgcolor = color.new(colDashBg, 0))
table.merge_cells(dash, 0, 0, 2, 0)

// ─── Row 1: Subtitle ───────────────────────────────────────
table.cell(dash, 0, 1, "SMC  ×  ORDER FLOW",
  text_color = colNeutralLt, text_size = dashTextSize,
  bgcolor = color.new(colDashBg, 0))
table.cell(dash, 1, 1, "", bgcolor = color.new(colDashBg, 0))
table.cell(dash, 2, 1, "", bgcolor = color.new(colDashBg, 0))
table.merge_cells(dash, 0, 1, 2, 1)

// ─── Row 2: Column Headers ─────────────────────────────────
table.cell(dash, 0, 2, "METRIC",
  text_color = colNeutralLt, text_size = dashTextSize,
  bgcolor = color.new(colDivider, 85))
table.cell(dash, 1, 2, "STATE",
  text_color = colNeutralLt, text_size = dashTextSize,
  bgcolor = color.new(colDivider, 85))
table.cell(dash, 2, 2, "DETAIL",
  text_color = colNeutralLt, text_size = dashTextSize,
  bgcolor = color.new(colDivider, 85))

// ─── Row 3: Market Structure ───────────────────────────────
table.cell(dash, 0, 3, "STRUCTURE",
  text_color = colTextLight, text_size = dashTextSize,
```

```
    bgcolor = color.new(colDashBg, 0))
  table.cell(dash, 1, 3, f_structIcon(structureTrend),
    text_color = f_structClr(structureTrend, BULL_PRIMARY, BEAR_PRIMARY, colNeutralLt),
    text_size = dashTextSize,
    bgcolor = color.new(colDashBg, 0))
  table.cell(dash, 2, 3,
    (bosUp or bosDown ? "BOS " : "") + (chochUp or chochDown ? "CHoCH" : ""),
    text_color = colNeutralLt, text_size = dashTextSize,
    bgcolor = color.new(colDashBg, 0))

  // ——— Row 4: EMA Trend
————————————————————————————————————————————————————————————————————————————————
————————————
  table.cell(dash, 0, 4, "EMA TREND",
    text_color = colTextLight, text_size = dashTextSize,
    bgcolor = color.new(colDashBg, 0))
  table.cell(dash, 1, 4, f_emaTxt(emaTrendBull, emaTrendBear),
    text_color = f_emaClr(emaTrendBull, emaTrendBear, BULL_PRIMARY, BEAR_PRIMARY,
colNeutralLt),
    text_size = dashTextSize,
    bgcolor = color.new(colDashBg, 0))
  table.cell(dash, 2, 4,
    priceAboveEma ? "PRICE > 200" : priceBelowEma ? "PRICE < 200" : "AT 200",
    text_color = priceAboveEma ? BULL_DIM : priceBelowEma ? BEAR_DIM : colNeutralLt,
    text_size = dashTextSize,
    bgcolor = color.new(colDashBg, 0))

  // ——— Row 5: Trend Cloud
————————————————————————————————————————————————————————————————————————————————
————————————
  table.cell(dash, 0, 5, "CLOUD",
    text_color = colTextLight, text_size = dashTextSize,
    bgcolor = color.new(colDashBg, 0))
  table.cell(dash, 1, 5, cloudBull ? "▲ BULLISH" : cloudBear ? "▼ BEARISH" : "— FLAT",
    text_color = cloudBull ? BULL_PRIMARY : cloudBear ? BEAR_PRIMARY : colNeutralLt,
    text_size = dashTextSize,
    bgcolor = color.new(colDashBg, 0))
  table.cell(dash, 2, 5, "", bgcolor = color.new(colDashBg, 0))
```

```
// ── Row 6: Divider ──────────────────────────────────────────────────────────────────────────
table.cell(dash, 0, 6, "──────────────────────────────────────────────",
  text_color = colDivider, text_size = dashTextSize,
  bgcolor = color.new(colDashBg, 0))
table.cell(dash, 1, 6, "", bgcolor = color.new(colDashBg, 0))
table.cell(dash, 2, 6, "", bgcolor = color.new(colDashBg, 0))
table.merge_cells(dash, 0, 6, 2, 6)

// ── Row 7: Rel Volume ───────────────────────────────────────────────────────────────────────
table.cell(dash, 0, 7, "REL VOLUME",
  text_color = colTextLight, text_size = dashTextSize,
  bgcolor = color.new(colDashBg, 0))
table.cell(dash, 1, 7, f_volState(relVol, volSpikeMulti),
  text_color = f_volClr(relVol, volSpikeMulti, BULL_BRIGHT, BULL_DIM, colNeutralLt),
  text_size = dashTextSize,
  bgcolor = color.new(colDashBg, 0))
table.cell(dash, 2, 7,
  str.tostring(math.round(relVol * 100) / 100, "#.##") + "x",
  text_color = f_volClr(relVol, volSpikeMulti, BULL_BRIGHT, BULL_DIM, colNeutralLt),
  text_size = dashTextSize,
  bgcolor = color.new(colDashBg, 0))

// ── Row 8: Delta Flow ───────────────────────────────────────────────────────────────────────
table.cell(dash, 0, 8, "DELTA FLOW",
  text_color = colTextLight, text_size = dashTextSize,
  bgcolor = color.new(colDashBg, 0))
table.cell(dash, 1, 8, f_deltaTxt(smoothDelta),
  text_color = f_deltaClr(smoothDelta, BULL_PRIMARY, BEAR_PRIMARY, colNeutralLt),
  text_size = dashTextSize,
  bgcolor = color.new(colDashBg, 0))
table.cell(dash, 2, 8,
```

```
    (deltaBullDiv ? "ACCUM" : deltaBearDiv ? "DISTR" : ""),
    text_color = deltaBullDiv ? BULL_BRIGHT : deltaBearDiv ? BEAR_BRIGHT : colNeutralLt,
    text_size = dashTextSize,
    bgcolor = color.new(colDashBg, 0))

  // ─── Row 9: Active Zones ──────────────────────────────────────────

  table.cell(dash, 0, 9, "ACTIVE ZONES",
    text_color = colTextLight, text_size = dashTextSize,
    bgcolor = color.new(colDashBg, 0))
  table.cell(dash, 1, 9,
    "OB: " + str.tostring(dashOBCount) + "  •  FVG: " + str.tostring(dashFVGCount),
    text_color = colNeutralLt, text_size = dashTextSize,
    bgcolor = color.new(colDashBg, 0))
  table.cell(dash, 2, 9,
    (priceInBullOB ? "IN OB▲" : priceInBearOB ? "IN OB▼" : "") +
    (priceInBullFVG ? " IN FVG▲" : priceInBearFVG ? " IN FVG▼" : ""),
    text_color = (priceInBullOB or priceInBullFVG) ? BULL_PRIMARY :
      (priceInBearOB or priceInBearFVG) ? BEAR_PRIMARY : colNeutralLt,
    text_size = dashTextSize,
    bgcolor = color.new(colDashBg, 0))

  // ─── Row 10: Divider ──────────────────────────────────────────────

  table.cell(dash, 0, 10, "──────────────────────────────────────",
    text_color = colDivider, text_size = dashTextSize,
    bgcolor = color.new(colDashBg, 0))
  table.cell(dash, 1, 10, "", bgcolor = color.new(colDashBg, 0))
  table.cell(dash, 2, 10, "", bgcolor = color.new(colDashBg, 0))
  table.merge_cells(dash, 0, 10, 2, 10)

  // ─── Row 11: Bull Score ───────────────────────────────────────────

  table.cell(dash, 0, 11, "BULL SCORE",
    text_color = colTextLight, text_size = dashTextSize,
```

```
    bgcolor = color.new(colDashBg, 0))
  table.cell(dash, 1, 11, f_scoreTxt(bullScore, bullConfidence),
    text_color = f_scoreClr(bullScore, minConfluence, BULL_PRIMARY, colNeutralLt),
    text_size = dashTextSize,
    bgcolor = color.new(colDashBg, 0))
  table.cell(dash, 2, 11, f_gradeFromConf(bullConfidence),
    text_color = f_gradeClr(bullConfidence, BULL_BRIGHT, BULL_DIM, colNeutralLt),
    text_size = dashTextSize,
    bgcolor = color.new(colDashBg, 0))

  // ── Row 12: Bear Score
──────────────────────────────────────────────────────────────────────────────────
──────────
  table.cell(dash, 0, 12, "BEAR SCORE",
    text_color = colTextLight, text_size = dashTextSize,
    bgcolor = color.new(colDashBg, 0))
  table.cell(dash, 1, 12, f_scoreTxt(bearScore, bearConfidence),
    text_color = f_scoreClr(bearScore, minConfluence, BEAR_PRIMARY, colNeutralLt),
    text_size = dashTextSize,
    bgcolor = color.new(colDashBg, 0))
  table.cell(dash, 2, 12, f_gradeFromConf(bearConfidence),
    text_color = f_gradeClr(bearConfidence, BEAR_BRIGHT, BEAR_DIM, colNeutralLt),
    text_size = dashTextSize,
    bgcolor = color.new(colDashBg, 0))

  // ── Row 13: Verdict
──────────────────────────────────────────────────────────────────────────────────
──────────────────
  table.cell(dash, 0, 13, "VERDICT",
    text_color = colTextLight, text_size = dashTextSize,
    bgcolor = color.new(colDashBg, 0))
  table.cell(dash, 1, 13, verdictTxt,
    text_color = verdictClr, text_size = dashTextSize,
    bgcolor = color.new(colDashBg, 0))
  table.cell(dash, 2, 13, "", bgcolor = color.new(colDashBg, 0))
  table.merge_cells(dash, 1, 13, 2, 13)

//
```

```
// ═══════════════════════════════════════════════ ALERT CONDITIONS ═══════════════════════════════════════════════
//
// ═══════════════════════════════════════════════════════════════════════════════════════════════════════════════

alertcondition(bullSignal,
  title   = "Nexus Long Signal",
  message = "AlphaX Nexus: LONG confluence signal on {{ticker}} {{interval}}")

alertcondition(bearSignal,
  title   = "Nexus Short Signal",
  message = "AlphaX Nexus: SHORT confluence signal on {{ticker}} {{interval}}")

alertcondition(bosUp,
  title   = "Bullish BOS",
  message = "AlphaX Nexus: Bullish Break of Structure on {{ticker}} {{interval}}")

alertcondition(bosDown,
  title   = "Bearish BOS",
  message = "AlphaX Nexus: Bearish Break of Structure on {{ticker}} {{interval}}")

alertcondition(chochUp,
  title   = "Bullish CHoCH",
  message = "AlphaX Nexus: Bullish Change of Character on {{ticker}} {{interval}}")

alertcondition(chochDown,
  title   = "Bearish CHoCH",
  message = "AlphaX Nexus: Bearish Change of Character on {{ticker}} {{interval}}")

alertcondition(bullLiqSweep,
  title   = "Bullish Liquidity Sweep",
  message = "AlphaX Nexus: Bullish liquidity sweep on {{ticker}} {{interval}}")

alertcondition(bearLiqSweep,
  title   = "Bearish Liquidity Sweep",
```

```
  message = "AlphaX Nexus: Bearish liquidity sweep on {{ticker}} {{interval}}")

alertcondition(bullAbsorption,
  title   = "Bullish Absorption",
  message = "AlphaX Nexus: Institutional buying absorption detected on {{ticker}} {{interval}}")

alertcondition(bearAbsorption,
  title   = "Bearish Absorption",
  message = "AlphaX Nexus: Institutional selling absorption detected on {{ticker}} {{interval}}")
```